

```

// Local-only SQLite store. Holds ONLY
camera node data (public OSM info).
// No user location is ever written to this
database.

import * as SQLite from 'expo-sqlite';

const DB_NAME = 'nodemap_cache.db';
let dbInstance = null;

export async function getDb() {
  if (dbInstance) return dbInstance;
  dbInstance = await
SQLite.openDatabaseAsync(DB_NAME);
  await dbInstance.execAsync(`
    PRAGMA journal_mode = WAL;
    CREATE TABLE IF NOT EXISTS camera_nodes
  (
    osm_id INTEGER PRIMARY KEY NOT NULL,
    lat REAL NOT NULL,
    lon REAL NOT NULL,
    surveillance_type TEXT,
    direction TEXT,
    last_synced_at INTEGER NOT NULL
  );
  CREATE INDEX IF NOT EXISTS
idx_camera_lat_lon ON camera_nodes (lat,
lon);
`);
  return dbInstance;
}

```

```

}

export async function
upsertCameraNodes(nodes) {
  const db = await getDb();
  const now = Date.now();
  await db.withTransactionAsync(async () =>
  {
    for (const n of nodes) {
      await db.runAsync(
        `INSERT INTO camera_nodes (osm_id,
lat, lon, surveillance_type, direction,
last_synced_at)
VALUES (?, ?, ?, ?, ?, ?)
ON CONFLICT(osm_id) DO UPDATE SET
lat = excluded.lat,
lon = excluded.lon,
surveillance_type =
excluded.surveillance_type,
direction = excluded.direction,
last_synced_at =
excluded.last_synced_at;`,
        [n.osmId, n.lat, n.lon,
n.surveillanceType ?? null, n.direction ??
null, now]
      );
    }
  });
}

// Returns cached nodes inside a bounding
box [minLat, minLon, maxLat, maxLon].
export async function
getCameraNodesInBounds(bounds) {

```

```
const db = await getDb();
const { minLat, minLon, maxLat, maxLon }
= bounds;
const rows = await db.getAllAsync(
  `SELECT osm_id as osmId, lat, lon,
surveillance_type as surveillanceType,
direction
  FROM camera_nodes
  WHERE lat BETWEEN ? AND ? AND lon
BETWEEN ? AND ?;`,
  [minLat, maxLat, minLon, maxLon]
);
return rows;
}
```